

Capacity Connect — Recommended Tech Stack

Detailed rationale, trade-offs and alternatives for each layer of the LMS build

FRONTEND FRAMEWORKNext.js 14 + TypeScript React-based, built-in routing & SSR, one framework covers UI and light backend (API routes). TypeScript catches bugs early — useful with 4 different role-based views (Trainee/Trainer/Admin/Visitor). PROS Fast dev speed, huge ecosystem, SEO-friendly (SSR), one repo for FE+BE possible CONS Steeper learning curve than plain React, opinionated routing/file structure Not using: Plain React+Vite (no SSR, needs separate backend), Vue/Nuxt (smaller ecosystem), Angular (heavier, overkill for hackathon)	BACKEND / APINode.js + Express (or Next.js API routes) Same language as frontend (JS/TS) — less context switching, one shared type system. REST fits the CRUD-heavy nature of courses/enrollments/forum threads well. PROS Fast to build, one language across stack, huge npm ecosystem, easy to deploy CONS Single-threaded (CPU-heavy tasks need workarounds), less structure than Django out of the box Not using: Django/Python (great for ML-heavy apps, but splits stack language), Spring Boot/Java (too heavy/slow for hackathon speed), GraphQL API (added complexity not needed for CRUD)	DATABASEPostgreSQL (via Supabase or Neon) Data is deeply relational: users → courses → enrollments → forum threads → replies → votes, all linked by foreign keys. Joins like "posts by users enrolled in course X" stay simple in SQL. Managed hosting (Supabase/Neon) means zero DB-ops work. PROS ACID compliance, strong relational integrity, generous free tier, Supabase bundles auth+storage+realtime too CONS Vertical scaling has limits vs. NoSQL horizontal scaling (not a concern at hackathon/early-product scale) Not using: MongoDB/NoSQL (nested replies + role-permission joins get messy in document stores), Firebase Firestore (weak relational queries, harder RBAC at data layer)
STYLING / UITailwind CSS + shadcn/ui Utility-first CSS = fast responsive UI without hand-writing stylesheets. shadcn/ui gives pre-built accessible components (tables, modals, forms) you copy in and customize — no design-from-scratch needed. PROS Very fast to build polished UI, fully customizable, no runtime CSS-in-JS overhead CONS HTML gets class-heavy/verbose, shadcn needs manual component updates Not using: Material UI (heavier, harder to restyle), Bootstrap (dated look), plain CSS/SASS (too slow to build with)	AUTHENTICATIONNextAuth.js (Auth.js) or Clerk Handles JWT/session management, role-based auth (Trainee/Trainer/Admin), and optional social login — without building auth from scratch, which is a common time-sink and security risk in hackathons. PROS Battle-tested security, saves days of dev time, easy RBAC middleware integration CONS Clerk has usage-based pricing at scale; less low-level control than custom JWT auth Not using: Custom JWT-from-scratch (more control, but higher security risk under time pressure), Firebase Auth (locks you into Firebase ecosystem)	ORMPrisma Type-safe database queries auto-generated from your schema — matches all core entities (users, courses, assessments, certificates, forum_threads, forum_replies, competency_map). Built-in migration system keeps schema versioned as it evolves. PROS Full TypeScript type-safety end-to-end, clean migration history, great docs/DX CONS Extra abstraction layer vs. raw SQL; complex joins sometimes need `\$.queryRaw` Not using: Sequelize (older, weaker TS support), raw SQL/pg driver (full control, but much slower to build with, error-prone under deadline)
STATE & DATA FETCHINGZustand + TanStack Query Zustand handles simple global state (auth/user role) with minimal boilerplate. TanStack Query handles server data — caching, loading/error states, auto-refetch — keeps dashboards feeling fast. PROS Tiny bundle size, far less boilerplate than Redux, great caching out of the box CONS Less structure/enforcement than Redux on very large teams Not using: Redux Toolkit (more boilerplate, overkill at this scale), Context API alone (no caching, re-render issues)	FILE / MEDIA UPLOADSUploadThing (or Multer + S3) Trainer Library needs PDF/slide/video uploads. UploadThing handles storage + CDN + type/size validation with almost no setup — ideal under hackathon time constraints. PROS Minimal setup, built-in CDN delivery, type/size validation out of the box CONS Free tier storage limits; less control than raw S3 configuration Not using: Raw AWS S3 + Multer (more control, much more setup time), Cloudinary (better for images specifically, less ideal for large video/slide files)	SEARCHPostgres full-text search (tsvector) Built directly into Postgres — no extra infrastructure needed for searching courses/forum threads by keyword/tag. Good enough for hackathon and early-product scale. PROS Zero extra service, already in your DB, free, fast enough at this scale CONS No typo-tolerance or semantic understanding like dedicated search engines Not using (for now): Meilisearch/Algolia (instant, typo-tolerant — good stretch goal), embedding-based semantic search (real scope commitment, not needed for MVP)
FORMS & VALIDATIONReact Hook Form + Zod Needed for profile forms, quiz creation, course creation. Zod schemas can be reused on the backend too — one validation source of truth for both client and server. PROS Minimal re-renders, type-safe validation, shareable schema FE/BE CONS Extra library to learn if team is new to schema validation Not using: Formik (more re-renders, larger bundle), plain HTML validation (not enough for complex role-based forms)	REAL-TIME (Forum notifications)Supabase Realtime (or Socket.io) Powers "someone replied to your question" live notifications without building a WebSocket server manually — comes free if you're already on Supabase for the database. PROS Zero extra infra if using Supabase, simple subscribe-to-table API CONS Vendor-tied if using Supabase's version; Socket.io needs its own server/scaling plan Not using: Polling only (simplest, but not truly live), Pusher/Ably (good but extra paid service)	DEPLOYMENTVercel (app) + Supabase/Neon (DB) Vercel pairs natively with Next.js — one-click deploy, free SSL/domain, generous free tier. Database is already hosted separately on Supabase/Neon, so there's zero server management required. PROS Zero DevOps, instant deploys on every git push, free tier covers hackathon + early demo needs CONS Serverless cold-starts on free tier; costs scale up with real production traffic Not using: Self-hosted VPS/DigitalOcean (full control, but requires DevOps time you don't have), AWS full stack (powerful but heavy setup for a hackathon timeline)

Capacity Connect — LMS Core Mechanics: Video & Quiz Layer

Tech specifically for course video playback and MCQ assessments — not covered by generic infra above

VIDEO PLAYER Vidstack (or Plyr.js) Raw HTML5 <video> tags look unstyled and give no hooks for progress-tracking events. Vidstack is a modern, accessible, fully customizable player component that fits a React/Next.js app directly and exposes clean event hooks (play, pause, timeupdate, ended). PROS Accessible out of the box, themeable to match your UI, framework-agnostic React bindings, small bundle CONS Newer library — smaller community than Video.js; some advanced plugins need custom work Not using: Video.js (older, heavier, jQuery-era API), raw <video> tag (no styling/progress hooks), YouTube/Vimeo embed (loses control over analytics + branding, needs external hosting)	QUIZ BUILDER UI React Hook Form + Zod (already in stack) Trainers create MCQ questions, options, correct answers and deadlines. No off-the-shelf quiz-builder library fits your exact role/permission model and forum-linked schema, so this is built as custom dynamic form fields (add/remove question, add/remove option). PROS Full control over question types/validation, reuses libraries already in the stack, no new dependency CONS More upfront dev time than a plug-in quiz library; you own all edge-case handling Not using: Typeform/Google Forms embed (can't enforce RBAC, can't tie results back into your competency-mapping schema), off-the-shelf quiz npm packages (rarely match custom deadline + role logic)
VIDEO HOSTING & STREAMING Cloudinary Video (or Mux) Trainer-uploaded lecture videos need to auto-transcode into adaptive bitrate streams so playback doesn't buffer on poor connections — directly serves the ministry's offline/low-bandwidth requirement for field staff (ships, remote stations). PROS Auto-transcoding + adaptive streaming, built-in CDN, thumbnail generation, easy SDK integration CONS Free tier has storage/bandwidth caps; costs rise with video volume & watch-hours at scale Not using: Raw S3 + <video> tag (no adaptive bitrate, buffers badly on slow connections), self-hosted FFmpeg transcoding pipeline (powerful but heavy DevOps lift for a hackathon timeline)	DEADLINE / TIMER LOGIC date-fns + Vercel Cron (or node-cron) Quizzes have deadlines (per the report's `assessments.deadline` field). date-fns handles clean deadline comparisons/countdown display; a scheduled job auto-locks quizzes past their deadline and can auto-submit in-progress attempts on timeout. PROS Lightweight, no heavy scheduling infra needed, Vercel Cron is free-tier friendly and zero-setup CONS Vercel Cron has a minimum interval (1 min) — not truly real-time to the second; self-hosted node-cron needs an always-on server Not using: Manual client-side-only timers (unreliable — user can manipulate local clock/tab), heavy job queues like BullMQ (overkill for simple deadline checks at this scale)
WATCH-PROGRESS TRACKING Custom onTimeUpdate → DB To mark a course "completed" (per the `enrollments.progress` / `completion_status` fields), the player emits periodic timeupdate events; the frontend batches and saves `watched_seconds` to the backend, which computes % complete against video duration. PROS Simple to build, no external service, gives you exact per-user progress data for dashboards CONS Naïve implementation can be gamed (seeking to the end); needs basic anti-skip logic for real completion tracking Not using: Third-party LMS analytics SaaS (e.g. Wistia stats) — overkill and adds vendor cost for what's a simple progress-percentage need	AUTO-GRADING Server-side comparison logic MCQs are objectively gradable — on submit, the backend compares submitted `option_id` per question against the stored `correct_option`, computes a score, and writes it to the `attempts` table. No AI or external grading service is needed for multiple-choice. PROS Instant results for trainees, zero cost, fully deterministic and auditable CONS Only works for objective question types (MCQ) — if you ever add short-answer/essay questions, you'd need manual review or an LLM grader Not using: Third-party assessment/proctoring SaaS (e.g. ExamSoft) — unnecessary cost/complexity for straightforward subject-wise MCQs
	SCORE & PARTICIPATION ANALYTICS Recharts (already in stack) Powers trainer dashboards: quiz score distribution, drop-off rate, average completion time, and participation trends — all explicit differentiator features from the report, all backed by the same `attempts` and `enrollments` tables. PROS Same library already used for admin dashboards — no new dependency, consistent chart styling app-wide CONS Less feature-rich than dedicated BI tools for very complex multi-dimensional analytics Not using: Chart.js (also fine, but Recharts fits React's component model more naturally), external BI tool like Metabase (extra infra, not needed at this scale)